Modern NLP Applications in Public Health

Alexander McLain

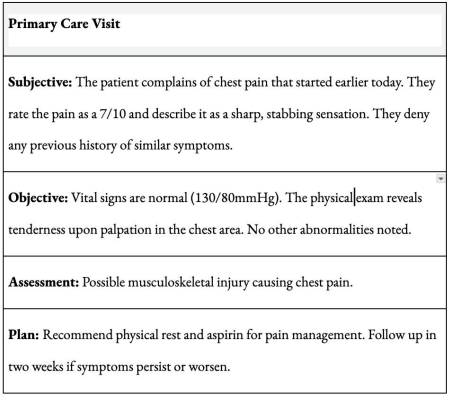
BIOS 735: Machine Learning for Public Health Applications

Where we are

- Last time: how an LLM works (tokens, embeddings, attention, next-token prediction) and how it is trained.
- Today: how to **use** text in a public health study.
  - What kinds of tasks come up?
  - What are the options, from word counts to LLMs?
  - How do we design, validate, and analyze a study that uses NLP?
- The main message: an NLP method is a **measurement tool**. The same rules apply as for any prediction model: labeled data, a held-out test set, and honest reporting.

Text data in public health

- Emergency department chief complaints and triage notes.
- Clinical notes: discharge summaries, radiology and pathology reports.
- Death certificates, coroner and law-enforcement narratives.
- Open-ended survey responses and interview transcripts.
- Social media posts, online reviews, news reports.
- The scientific literature itself.



Primary Care Visit

Subjective: The patient complains of chest pain that started earlier today. They rate the pain as a 7/10 and describe it as a sharp, stabbing sensation. They deny any previous history of similar symptoms.

Objective: Vital signs are normal (130/80mmHg). The physical exam reveals tenderness upon palpation in the chest area. No other abnormalities noted.

Assessment: Possible musculoskeletal injury causing chest pain.

Plan: Recommend physical rest and aspirin for pain management. Follow up in two weeks if symptoms persist or worsen.

Why clinical text is hard

Feature	Example
Abbreviations	"pt w/ SOB, hx of MI"
Misspellings	"diabetis", "pnuemonia"
Negation	"**denies** chest pain", "**no** evidence of fracture"
Uncertainty	"**possible** pneumonia", "r/o PE" (rule out)
Someone else	"**mother** had breast cancer"
Time	"**history of** stroke" vs. "acute stroke"
Copy and paste	the same template text in every note

- A keyword search for "pneumonia" finds all of these: "pneumonia", "no pneumonia", "r/o pneumonia", "mother had pneumonia".
- "SOB" means shortness of breath in a note, but something else on social media.

Common NLP tasks

Task	Public health example
Document classification	Is this ED visit an opioid overdose? Is this abstract relevant to my review?
Information extraction	Pull smoking status, pack-years, or housing instability out of a note.
Normalization (coding)	Map "heart attack", "MI", "STEMI" to one code (ICD-10, SNOMED).
Topic discovery	What themes appear in 5,000 open-ended survey responses?
Semantic search	Find all guideline passages about RSV vaccination in older adults.
Summarization	Summarize interview transcripts for qualitative analysis.

- The first three are **supervised** (there is a right answer); topic discovery is **unsupervised**.
- Most published public health NLP studies are classification or extraction.

Four ways to turn text into an answer

1. **Bag of words + a model we know:** count words, fit penalized logistic regression or a random forest.
2. **Embeddings as features:** turn each document into a vector with a pretrained model, then fit a model we know.
3. **Prompt an LLM:** describe the task (zero-shot) or give a few examples (few-shot); no training.
4. **Fine-tune a pretrained model:** train a BERT-type model on your labeled data.

Four ways to turn text into an answer

1. **Bag of words + a model we know:** count words, fit penalized logistic regression or a random forest.
2. **Embeddings as features:** turn each document into a vector with a pretrained model, then fit a model we know.
3. **Prompt an LLM:** describe the task (zero-shot) or give a few examples (few-shot); no training.
4. **Fine-tune a pretrained model:** train a BERT-type model on your labeled data.

- These go roughly from cheap, transparent, and local to flexible, expensive, and opaque.
- **All four need labeled data**: options 1, 2, and 4 to train; every option to test.

The bag-of-words pipeline

1. **Clean:** lower case, remove punctuation and numbers (maybe).
2. **Tokenize:** split into words; often add **bigrams** ("chest pain", "no fever").
3. **Reduce the vocabulary:** drop stop words ("the", "of"); drop rare words; **stemming** ("falls", "fell", "falling" $\rightarrow$ "fall").
4. **Document-term matrix:** one row per document, one column per word; entries are counts.
5. **Weight:** TF-IDF down-weights words that appear in almost every document:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log \frac{N}{\text{df}(t)},$$

   where $\text{tf}(t, d)$ is the count of term t in document d, N the number of documents, and $\text{df}(t)$ the number of documents that contain t.
6. **Model:** the matrix is $\boldsymbol{X}$; fit lasso/elastic net logistic regression.

Bag of words: pros and cons

Pros

- Uses tools we already know (`glmnet`, `ranger`).
- The lasso picks out words: easy to explain and to check.
- Runs on a laptop; the text never leaves your computer.
- Free, fast, and fully reproducible.
- A strong **baseline**; it is often hard to beat by much.

Cons

- "Fever" and "febrile" are unrelated columns.
- Word order is lost: "no fever, cough" = "fever, no cough".
- Very wide and sparse $\boldsymbol{X}$ (often $p \gg n$).
- Misspellings and abbreviations each need their own column.
- Needs a few hundred labeled documents or more.

Class example: fit TF-IDF + lasso to classify notes, and plot the words with the largest coefficients.

Handling negation: rules

- Before machine learning, clinical NLP used **rules**, and they are still common.
- **NegEx** (Chapman et al., 2001): a list of trigger phrases ("no", "denies", "negative for", "without") and a window of words they apply to.
- **ConText** (Harkema et al., 2009) adds two more attributes: **who** (patient or family member) and **when** (current or historical).

Sentence	Negated	Experiencer	Time
"Patient denies chest pain."	yes	patient	current
"Mother had breast cancer."	no	family	historical
"History of stroke in 2019."	no	patient	historical
"No fever, no cough."	yes	patient	current

Handling negation: rules

- Before machine learning, clinical NLP used **rules**, and they are still common.
- **NegEx** (Chapman et al., 2001): a list of trigger phrases ("no", "denies", "negative for", "without") and a window of words they apply to.
- **ConText** (Harkema et al., 2009) adds two more attributes: **who** (patient or family member) and **when** (current or historical).

Sentence	Negated	Experiencer	Time
"Patient denies chest pain."	yes	patient	current
"Mother had breast cancer."	no	family	historical
"History of stroke in 2019."	no	patient	historical
"No fever, no cough."	yes	patient	current

Discussion: What does NegEx do with "The patient was **not** sure whether she had a fever"?

Document embeddings

- An **embedding model** turns a whole sentence or document into one vector, typically with 384 to 3,000 numbers.
- Unlike our Austen word vectors, these are **contextual**: they come out of a Transformer, so "no fever" and "fever" get different vectors.
- Options:
  - Open models you can run locally (e.g., the `sentence-transformers` family; BioBERT, ClinicalBERT).
  - Commercial embedding APIs (OpenAI, Google, and others).
- Then use the vectors like any other features:
  - $\boldsymbol{X}$ = embeddings → penalized logistic regression or a random forest.
  - Cosine similarity → semantic search ("find notes like this one").
  - Clustering → themes in free text.

Embeddings as features: pros and cons

Pros

- Synonyms and misspellings land close together.
- A short, dense $\boldsymbol{X}$ (no $p \gg n$ problem).
- Works with fewer labels than bag of words: the embedding model already "knows" language.
- Cheap: one pass per document.

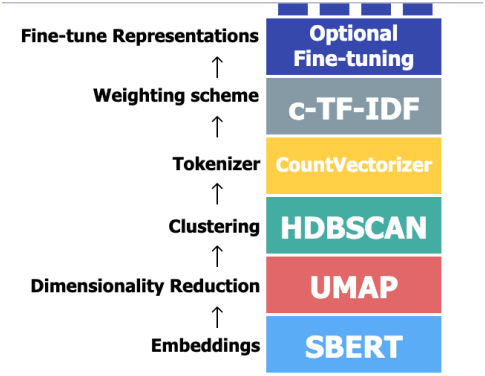
Cons

- The coefficients are on embedding dimensions, which have no meaning.
- A general-purpose model may not know clinical shorthand.
- With a commercial API, the text leaves your institution.
- Long documents may need to be split into chunks.

Class example: compare embeddings + lasso with TF-IDF + lasso on the same train/test split.

Finding themes: from topic models to embeddings

- **Topic models** (LDA): each document is a mixture of topics; each topic is a distribution over words. Built on bag of words.
- **Embedding-based** (e.g., BERTopic, Grootendorst 2022):
  1. embed each document,
  2. reduce the dimension (UMAP),
  3. cluster (e.g., HDBSCAN or k-means),
  4. describe each cluster by its most distinctive words, or ask an LLM to name it.
- Steps 2–3 are from our dimension reduction and clustering lectures.



An LLM as an annotator

- The idea: give the LLM the same instructions you would give a trained human coder.
- A labeling pipeline:
  1. Write a **codebook**: definitions, inclusion/exclusion rules, hard examples.
  2. Turn the codebook into a **prompt**.
  3. Ask for **structured output** (fixed labels or JSON) so it can be read into `R` .
  4. Run on a **development set**; compare with human labels; revise the prompt.
  5. Freeze the prompt and model version; run **once** on the test set.
  6. Run on all documents.
- Steps 4 and 5 are the same train/validate/test logic we use for tuning parameters: the prompt is a tuning parameter.

Information extraction with structured output

- Ask for several fields at once, each with a fixed set of allowed values.
- Always allow "not mentioned" or "unclear", or the model will guess.
- Ask for the **evidence**: the exact text the answer came from. This makes checking much faster.
- Many APIs can enforce a JSON schema, so the output always parses.

Note

"45 yo M w/ hx DM2, HTN. Quit smoking 5 yrs ago (20 pk-yr). Staying with sister since eviction last month."

Model output

```
{
  "smoking_status": "former",
  "pack_years": 20,
  "housing_instability": "yes",
  "evidence_housing": "Staying with sister
                       since eviction"
}
```

Calling an LLM from R

- The `ellmer` package gives one interface to many providers (OpenAI, Anthropic, Google, and local models through Ollama).
- `chat_structured()` returns the answer as an R object, following the type you define.
- Many documents: `parallel_chat_structured()` returns a data frame.
- API keys go in `.Renviron`, never in your code.

```
library(ellmer)

chat <- chat_ollama(           # local model
  model = "llama3.1",
  system_prompt = codebook)    # your codebook

fall_type <- type_object(
  fall     = type_enum(c("yes", "no", "unclear")),
  evidence = type_string()
)

chat$chat_structured(
  "Slipped on ice outside home, hit head",
  type = fall_type)
```

Class example: label a set of notes with an LLM and compare with the lasso.

Practical prompt design for annotation

- Put the codebook definitions in the prompt, word for word.
- Include a few **hard** examples (few-shot), not just easy ones.
- One document per request: batching many documents into one prompt can change the answers.
- Set the temperature to 0 (or as low as possible) and record it.
- Re-run a subset to check agreement between runs.
- Watch for **order effects**: in multiple-choice prompts, the first option can be chosen more often.
- Asking the model for a confidence score ("how sure are you, 0–100?") does not give a calibrated probability.

What will it cost?

- Cost is charged per token, with separate prices for input and output tokens:

$$\text{cost} \approx N_{\text{docs}} \times (\text{prompt tokens} + \text{document tokens} + \text{output tokens}) \times \text{price per token}.$$

- Example: 50,000 survey responses, a 400-token codebook prompt, 100-token responses, 20-token answers:

$$50{,}000 \times 520 = 26 \text{ million tokens}.$$

- Notice that the **prompt is most of the cost**. Many providers discount a prompt that is repeated ("prompt caching") and jobs that can wait a day ("batch" pricing).
- Prices change often; check the provider's current price list.
- A local open-weight model costs nothing per token, but needs a good computer, and is usually slower and less accurate.

Using an LLM to make training labels

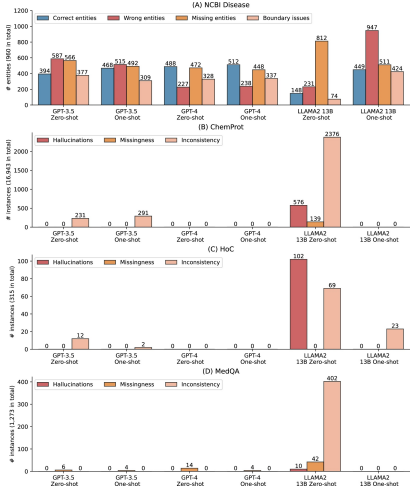
- A useful hybrid:
  1. Validate an LLM on a few hundred human-labeled documents.
  2. Use it to label tens of thousands of documents.
  3. Train a small, cheap model (bag of words or embeddings + lasso) on the LLM labels.
  4. Evaluate the small model against the **human** labels, not the LLM labels.
- The small model is fast, runs locally, and its predictions don't change when a commercial model is updated.
- It also inherits the LLM's mistakes, so step 4 is essential.

Fine-tuning a BERT-type model

- Start from an encoder model pretrained on biomedical or clinical text:
  - **BioBERT**, **PubMedBERT** (PubMed abstracts and papers),
  - **ClinicalBERT** (MIMIC intensive care notes).
- Add a classification layer (a softmax) and train on your labeled data; all of the weights are updated a little.
- Typically needs several hundred to a few thousand labeled documents and a GPU.
- Usually done in Python (Hugging Face `transformers`); from `R` through `reticulate`.
- The result is a model you own: it runs locally and never changes.

Fine-tuned models vs. prompted LLMs

- Benchmarks on biomedical NLP tasks (e.g., Chen et al., 2025):
  - For **extraction and classification**, fine-tuned smaller models often match or beat zero- and few-shot LLMs.
  - For **reasoning** tasks (e.g., medical question answering), large LLMs do better.
- With no labeled training data, prompting is the only one of the two that is available.
- The best choice depends on your task and data, so compare on **your** test set.



Choosing an approach

	Bag of words + lasso	Embeddings + lasso	Prompted LLM	Fine-tuned BERT
Labeled data to train	100s+	100s	none	100s–1,000s
Labeled data to test			**always**	
Runs locally	yes	possible	open models only	yes
Interpretable	yes	no	partly (evidence)	no
Handles negation, synonyms	poorly	partly	well	well
Cost per document	≈ 0	low	highest	low
Changes over time	no	no	yes (API)	no

Choosing an approach

	Bag of words + lasso	Embeddings + lasso	Prompted LLM	Fine-tuned BERT
Labeled data to train	100s+	100s	none	100s–1,000s
Labeled data to test			**always**	
Runs locally	yes	possible	open models only	yes
Interpretable	yes	no	partly (evidence)	no
Handles negation, synonyms	poorly	partly	well	well
Cost per document	≈ 0	low	highest	low
Changes over time	no	no	yes (API)	no

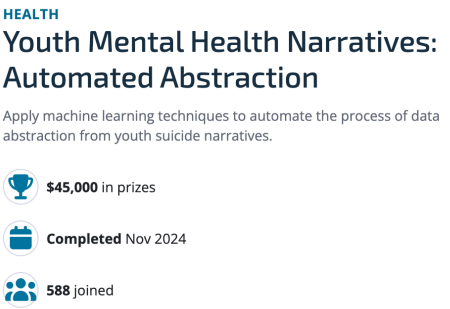
Rule of thumb: always fit the bag-of-words baseline. Use a more complex method only if it is clearly better **on the test set**.

Syndromic surveillance

- CDC's National Syndromic Surveillance Program (NSSP) gets ED visit data from most US EDs within about a day.
- Free-text **chief complaints** ("OD on heroin", "fell off ladder") are searched with keyword definitions.
- Used for overdoses, heat illness, influenza-like illness, firearm injuries, ...
- Challenges: very short text, misspellings, new slang ("tranq" for xylazine).

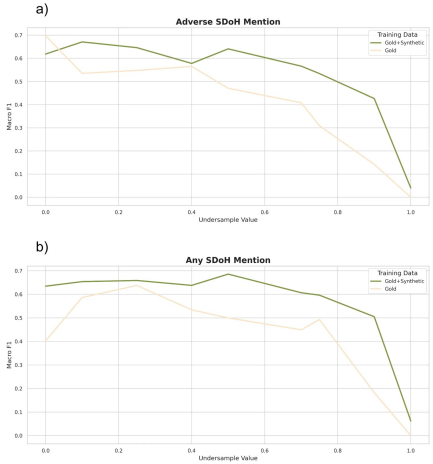
Coding death narratives

- The National Violent Death Reporting System (NVDRS) collects coroner and police **narratives** for suicides and homicides.
- Abstractors code dozens of circumstances from them (e.g., depressed mood, recent breakup).
- Slow, expensive, and coding varies between states.
- NLP can code these or flag cases for review; a 2024 public data challenge did this for youth suicides.



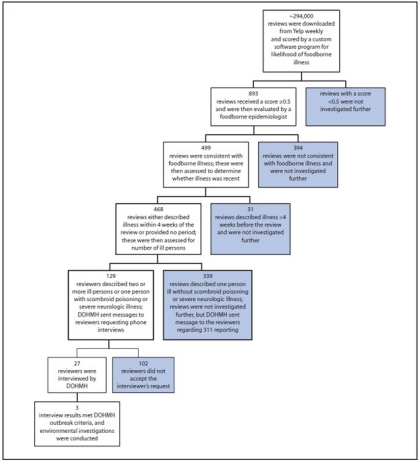
Social determinants of health from the EHR

- Housing, employment, transportation, and social support are rarely coded in the EHR, but often written in notes.
- Guevara et al. (2024) fine-tuned language models to find six social determinants in clinical notes:
  - the models found far more patients with an adverse social determinant than the ICD-10 Z-codes did;
  - they also checked whether predictions changed when race or gender words in the notes were changed.
- Note the two checks: performance against human labels, and **bias**.



Foodborne illness from online reviews

- New York City's health department screened Yelp restaurant reviews for reports of food poisoning (Harrison et al., 2014).
- A text classifier flagged reviews; epidemiologists read the flagged reviews and followed up.
- This found outbreaks that had not been reported to the health department.
- A typical design: the model **prioritizes**, and people make the decisions. So **sensitivity** and the **number to review** matter more than accuracy.



Screening for systematic reviews

- A review may start with 10,000 abstracts, of which 50 are relevant.
- **Active learning** (e.g., ASReview): a classifier ranks the abstracts, a person screens the top ones, the model refits, ...
- Most relevant papers are found after screening a small fraction.
- LLMs can also screen using the inclusion criteria as the prompt.
- **Missing** a study is the costly error: report recall, and set a stopping rule in advance.

Open-ended survey responses

- "Is there anything else you'd like to tell us about your health care?" with 5,000 answers.
- Traditional approach: two people code the answers with a codebook and check agreement (Cohen's κ).
- NLP options:
  - **Known categories:** an LLM codes with the codebook, a person codes a random sample, and we report κ between the LLM and the person, just as between two people.
  - **Unknown categories:** embed and cluster the answers, read examples from each cluster, then write the codebook.
- Responses may be in several languages: check performance in each language.

Class example: embed and cluster a set of free-text responses, and name the clusters.

How many documents do we need to label?

- To estimate sensitivity $\pm E$ with 95% confidence, we need about

$$n_+ = \frac{1.96^2 \, \mathrm{Se}(1 - \mathrm{Se})}{E^2}$$

  positive documents.

- Example: Se ≈ 0.90, $E = 0.05$: $n_+ = 139$ positive documents.
- If only 2% of documents are positive, a simple random sample would need $139/0.02 \approx 7{,}000$ documents to label.
- **Fix:** sample with **stratification**. Over-sample documents likely to be positive (e.g., they contain a keyword), sample the rest too, then weight back to the population when estimating sensitivity, specificity, and PPV.

How many documents do we need to label?

- To estimate sensitivity $\pm E$ with 95% confidence, we need about

$$n_+ = \frac{1.96^2 \, \mathrm{Se}(1 - \mathrm{Se})}{E^2}$$

  positive documents.

- Example: Se ≈ 0.90, $E = 0.05$: $n_+ = 139$ positive documents.
- If only 2% of documents are positive, a simple random sample would need $139/0.02 \approx 7{,}000$ documents to label.
- **Fix:** sample with **stratification**. Over-sample documents likely to be positive (e.g., they contain a keyword), sample the rest too, then weight back to the population when estimating sensitivity, specificity, and PPV.

Discussion: Why must we still sample from the documents *without* the keyword?

Building the gold standard

- Write the codebook **before** looking at model output.
- Have two people label the same subset independently; report agreement (κ); settle disagreements (adjudication).
- The model can't be expected to agree with the gold standard more than two people agree with each other.
- Keep the test set locked away: don't look at it while revising the prompt or the features.
- Label documents from every setting the model will be used in (hospitals, years, languages).

Using NLP labels in an analysis

- Suppose an LLM labels each ED visit as fall-related, and we want the proportion of visits that are falls.
- Misclassification biases the naive proportion $\hat{p}$. If Se and Sp are known, the Rogan–Gladen correction is

$$\hat{\pi} = \frac{\hat{p} + \text{Sp} - 1}{\text{Se} + \text{Sp} - 1}.$$

- Example: $\hat{p} = 0.12$, $\text{Se} = 0.85$, $\text{Sp} = 0.95$ gives $\hat{\pi} = 0.07/0.80 \approx 0.088$. The naive estimate is 37% too high.
- But Se and Sp are **estimated** from the validation sample, so their uncertainty must go into the confidence interval.
- If Se and Sp differ between groups (e.g., languages), comparisons between the groups are biased too.

Things change over time

- **The text changes:** new terms ("COVID-19", "xylazine"), new note templates, a new EHR system.
- **The population changes:** a new hospital joins the surveillance system.
- **The model changes:** commercial LLMs are updated and retired, sometimes with little notice.
- So:
  - Pin a specific model version, and record it with the prompt and date.
  - Re-check performance on a fresh labeled sample at regular intervals.
  - Watch for sudden jumps in the proportion of positive labels; it may be the model, not the epidemic.

Before you start: a checklist

- **Data use:** Is the text protected health information? Does the data use agreement or IRB approval allow sending it to an outside company? If not, use local models or an institution-approved service (with a business associate agreement).
- **Baseline:** What does a keyword search or bag of words + lasso achieve?
- **Labels:** Is there a codebook, a gold standard with inter-rater agreement, and a locked test set?
- **Metrics:** Which error matters more? Report sensitivity, PPV, and F1 with confidence intervals, overall and by subgroup.
- **Analysis:** Are downstream estimates corrected for misclassification?
- **Reporting:** model name and version, date, prompt, settings, and code.

Summary

- Public health has a lot of text; most applications are **classification** or **extraction**.
- Four approaches: bag of words, embeddings as features, prompting an LLM, and fine-tuning. Start with the simplest and only move on if it clearly helps.
- Clinical text has its own problems: abbreviations, negation, family history, and time.
- An NLP tool is a **measurement instrument**: build a gold standard, sample carefully, and account for its errors in the analysis.
- In the class example we will compare TF-IDF + lasso, embeddings + lasso, and a prompted LLM on the same labeled data, and cluster free-text responses.

References I

- Chapman WW, et al. (2001). A simple algorithm for identifying negated findings and diseases in discharge summaries. *J Biomed Inform*.
- Harkema H, et al. (2009). ConText: An algorithm for determining negation, experiencer, and temporal status from clinical reports. *J Biomed Inform*.
- Grootendorst M (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv:2203.05794*.
- Chen Q, et al. (2025). Benchmarking large language models for biomedical natural language processing applications and recommendations. *Nature Communications*.
- Guevara M, et al. (2024). Large language models to identify social determinants of health in electronic health records. *npj Digital Medicine*.
- Harrison C, et al. (2014). Using online reviews by restaurant patrons to identify unreported cases of foodborne illness — New York City, 2012–2013. *MMWR* 63(20).

References II

- van de Schoot R, et al. (2021). An open source machine learning framework for efficient and transparent systematic reviews. *Nature Machine Intelligence*.
- Rogan WJ, Gladen B (1978). Estimating prevalence from the results of a screening test. *Am J Epidemiol*.
- Angelopoulos AN, et al. (2023). Prediction-powered inference. *Science*.
- Silge J, Robinson D. *Text Mining with R*. tidytextmining.com